

```

"""
Tests for the abstract :class:`src.sensors.SensorNode` base class.

Since ``SensorNode`` is abstract, a minimal concrete ``DummySensor``
subclass is defined locally for testing purposes only.
"""

import pytest

from src.exceptions import CalibrationError, SensorOfflineError
from src.sensors import SensorNode

class DummySensor(SensorNode):
    """Minimal concrete SensorNode subclass used only for testing."""

    @property
    def sensor_id(self) -> str:
        """str: The unique identifier of this dummy sensor."""
        return self._sensor_id

    def read(self):
        """Return a fixed dummy value plus the current calibration offset."""
        return 0.0 + self.calibration_offset

# -----
# sensor_id validation
# -----

def test_valid_sensor_id_is_accepted():
    """A correctly formatted sensor_id should construct without error."""
    sensor = DummySensor(
        sensor_id="SN-TEMP-01",
        location="Building A",
        unit="C",
        safe_min=-20.0,
        safe_max=80.0,
    )
    assert sensor.sensor_id == "SN-TEMP-01"

@pytest.mark.parametrize(

```

```

    "bad_id",
    [
        "SN-TEMP-1",      # only 1 digit instead of 2
        "SN-TEMP-001",    # 3 digits instead of 2
        "SN-TMP-01",      # only 3 letters in type code
        "SN-TEMP-01",     # 5 letters in type code
        "sn-temp-01",     # lowercase
        "TEMP-01",        # missing "SN-" prefix
        "SN-TE2P-01",     # digit inside type code
        "",               # empty string
    ],
)

def test_invalid_sensor_id_raises_value_error(bad_id):
    """Malformed sensor_id strings should raise ValueError on construction."""
    with pytest.raises(ValueError):
        DummySensor(
            sensor_id=bad_id,
            location="Building A",
            unit="C",
            safe_min=-20.0,
            safe_max=80.0,
        )

# -----
# calibration_offset validation
# -----

def test_calibration_offset_within_range_is_accepted():
    """An offset well within +/-10% of full scale should be accepted."""
    # full scale = 100, 10% = 10
    sensor = DummySensor(
        sensor_id="SN-PRES-02",
        location="Building B",
        unit="kPa",
        safe_min=0.0,
        safe_max=100.0,
        calibration_offset=5.0,
    )
    assert sensor.calibration_offset == 5.0

def test_calibration_offset_boundary_values_are_accepted():
    """Offsets at exactly +10% and -10% of full scale should be accepted."""
    # full scale = 100, 10% = 10, so +/-10.0 is the inclusive boundary
    sensor = DummySensor(

```

```

        sensor_id="SN-PRES-03",
        location="Building B",
        unit="kPa",
        safe_min=0.0,
        safe_max=100.0,
    )

    sensor.calibrate(10.0)
    assert sensor.calibration_offset == 10.0

    sensor.calibrate(-10.0)
    assert sensor.calibration_offset == -10.0

def test_calibration_offset_outside_range_raises_calibration_error():
    """An offset just beyond +/-10% of full scale should raise CalibrationError."
    # full scale = 100, 10% = 10, so 10.01 should be rejected
    sensor = DummySensor(
        sensor_id="SN-PRES-04",
        location="Building B",
        unit="kPa",
        safe_min=0.0,
        safe_max=100.0,
    )

    with pytest.raises(CalibrationError):
        sensor.calibrate(10.01)

    with pytest.raises(CalibrationError):
        sensor.calibrate(-10.01)

def test_invalid_initial_calibration_offset_raises_calibration_error():
    """An out-of-range calibration_offset passed to __init__ should raise."""
    with pytest.raises(CalibrationError):
        DummySensor(
            sensor_id="SN-FLOW-01",
            location="Building C",
            unit="L/min",
            safe_min=0.0,
            safe_max=50.0,
            calibration_offset=10.0, # 10% of 50 is 5, so this exceeds it
        )

# -----
# Abstract class behavior

```

```

# -----

def test_sensor_node_cannot_be_instantiated_directly():
    """SensorNode is abstract and must not be directly instantiable."""
    with pytest.raises(TypeError):
        SensorNode(
            sensor_id="SN-TEMP-01",
            location="Building A",
            unit="C",
            safe_min=-20.0,
            safe_max=80.0,
        )

# -----
# self_test behavior
# -----

def test_self_test_passes_for_valid_calibration():
    """self_test should return True when calibration_offset is in range."""
    sensor = DummySensor(
        sensor_id="SN-VIBR-01",
        location="Building D",
        unit="mm/s",
        safe_min=0.0,
        safe_max=20.0,
        calibration_offset=1.0,
    )
    assert sensor.self_test() is True

def test_self_test_raises_sensor_offline_error_when_offset_drifted():
    """self_test should raise SensorOfflineError if offset has drifted out of range"""
    sensor = DummySensor(
        sensor_id="SN-GASC-01",
        location="Building E",
        unit="ppm",
        safe_min=0.0,
        safe_max=1000.0,
    )
    # Bypass the property's validation to simulate post-construction drift
    # (e.g. hardware degradation) that self_test should catch.
    sensor._calibration_offset = 999.0

    with pytest.raises(SensorOfflineError):

```

```
        sensor.self_test()

# -----
# __str__ / __repr__ sanity checks
# -----

def test_str_and_repr_contain_sensor_id():
    """__str__ and __repr__ should both include the sensor_id for debuggability."""
    sensor = DummySensor(
        sensor_id="SN-FLOW-02",
        location="Building F",
        unit="L/min",
        safe_min=0.0,
        safe_max=50.0,
    )
    assert "SN-FLOW-02" in str(sensor)
    assert "SN-FLOW-02" in repr(sensor)
```